

**‘Optimisation Programming Language Coursework – Problem
Summaries’**



Andrew Wilson (2417549)

MSc Business Analytics and Marketing

BU52711 – Prescriptive Analytics

DATE: 25/03/2026

Word Count: 1,768

Contents

1.0 Problem 1: The Dietitian's Problem	1
1.1 Problem Overview	1
1.2 Mathematical Formula	1
1.2.1 Decision Variables	1
1.2.2 Objective Function – Minimise Cost per Serving	1
1.2.3 Constraints	1
1.3 OPL Model Code	2
1.3.1 Dietitian.mod	3
1.3.2 Dietitian.dat	3
1.4 Food Item Data	4
1.5 Optimal Results	4
1.6 Interpretation of Results	5
1.7 Sensitivity Analysis	6
1.7.1 Reduced Costs (from CPLEX Output)	6
2.0 Problem 2: Dr J's Assignment Problem	7
2.1 Problem Overview	7
2.2 Mathematical Formula	7
1.2.1 Decision Variables	7
1.2.2 Objective Function – Minimise Total Preference Rank	7
1.2.3 Subject to	7
2.3 OPL Model Code	7
2.3.1 Assignment.mod	8
2.3.2 Assignment.dat	8
2.4 Optimal Assignment Results	9
2.5 Summary Statistics	11
2.6 Project Group Composition	11
2.7 Interpretation of Results	11
3.0 Problem 3: Shortest Time to Walk	12
3.1 Problem Overview	12
3.2 Network Structure	12
3.3 Mathematical Formula	13
3.3.1 Decision Variables	13
3.3.2 Objective Function – Minimise Total Walking Time	13
3.3.3 Flow Balance Constraints	14
3.4 OPL Model Code	14

3.4.1 Shortestpath.mod	15
3.4.2 Shortestpath.dat	15
3.5 Optimal Solution.....	17
3.6 Interpretation of Results	17

1.0 Problem 1: The Dietitian's Problem

1.1 Problem Overview

Sam Michuta is a dietitian for the University football team and is tasked with designing a minimum-cost lunch menu that must simultaneously meet six nutritional guidelines. There are seven food items that are available, that are each characterised by nutrient content per pound (lbs) and cost per pound (\$). This is a continuous Linear Program (LP) problem where decision variables represent non-negative real quantities of each food, and the model must determine what the optimal solution to the minimum-cost menu is.

1.2 Mathematical Formula

1.2.1 Decision Variables

$x \geq 0$ denotes the pounds of food item j selected per serving, where $j \in \{\text{Chicken, Fish, Ground Beef, Fried Beans, Lettuce, Potatoes, Milk 2\%}\}$.

1.2.2 Objective Function – Minimise Cost per Serving

Minimise $Z = 0.80x_1 + 3.70x_2 + 2.30x_3 + 0.90x_4 + 0.75x_5 + 0.40x_6 + 0.83x_7$

1.2.3 Constraints

Calories (kcal): $1,500 \leq 520x_1 + 500x_2 + 860x_3 + 600x_4 + 50x_5 + 460x_6 + 240x_7 \leq 2,000$

Iron (mg): $4.4x_1 + 3.3x_2 + 0.3x_3 + 3.4x_4 + 0.5x_5 + 2.2x_6 + 0.2x_7 \geq 5$

Fat (g): $20 \leq 30x_1 + 5x_2 + 75x_3 + 3x_4 + 10x_7 \leq 60$

Protein (g): $17x_1 + 85x_2 + 82x_3 + 10x_4 + 6x_5 + 10x_6 + 16x_7 \geq 30$

Carbohydrates (g): $30x_4 + 70x_6 + 22x_7 \geq 40$

Cholesterol (mg): $180x_1 + 90x_2 + 350x_3 + 20x_7 \leq 30$

Non-negativity: $x \geq 0$ for all j

1.3 OPL Model Code

The model uses {string} sets and float arrays to ensure it remains fully general with no data being hard-coded into the .mod file. The data is being read directly from an Excel spreadsheet via SheetConnection featured in the .dat file.

1.3.1 Dietitian.mod

```
/******
 * OPL 22.1.2.0 Model
 * Author: wilso
 * Creation Date: Mar 22, 2026 at 11:11:35 AM
 *****/

//sets and parameters
{string} F = ...; //set of food items, indexed by j
{string} N = ...; //set of nutrients, indexed by i
float a[N][F] = ...; //nutrient i content per pound of food j
float L[N] = ...; //lower bound on nutrient i per serving
float U[N] = ...; //upper bound on nutrient i per serving
float c[F] = ...; //cost per pound of food j

//decision variables
dvar float+ x[F]; //pounds of food item j selected (>= 0)

//objective function - minimise total cost per serving
minimize sum(j in F) c[j] * x[j];

//constraints
subject to {
    //lower bound on each nutrient
    forall(i in N)
        ctLower: sum(j in F) a[i][j] * x[j] >= L[i];

    //upper bound on each nutrient
    forall(i in N)
        ctUpper: sum(j in F) a[i][j] * x[j] <= U[i];
}
```

1.3.2 Dietitian.dat

```
/******
 * OPL 22.1.2.0 Data
 * Author: wilso
 * Creation Date: Mar 22, 2026 at 11:11:35 AM
 *****/

// The data is read directly from an Excel sheet as required by the question.
// Named ranges are used instead of hard-coded cell addresses so the
// .dat file remains valid, even if rows or columns are added to the spreadsheet.

SheetConnection data("dietitian_data.xlsx");

F from SheetRead(data, "Sheet1!foods"); //food item names (row 2, cols C:I)
N from SheetRead(data, "Sheet1!nutrients"); //nutrient names (col B, rows 3:8)
a from SheetRead(data, "Sheet1!content"); //content matrix (rows 3:8, cols C:I)
c from SheetRead(data, "Sheet1!cost"); //cost per lb (row 9, cols C:I)
L from SheetRead(data, "Sheet1!lower"); //lower bound/nutrient (col J, rows 3:8)
U from SheetRead(data, "Sheet1!upper"); //upper bound/nutrient (col K, rows 3:8)
```

1.4 Food Item Data

Table 1 presents the full nutritional profile for each of the food items, with the items selected for the optimal solution being highlighted in green and those excluded shown as “-”.

Food Item	Calories (kcal)	Iron (mg)	Protein (g)	Carbohydrates (g)	Fat (g)	Cholesterol (mg)	Cost (\$/lbs)	X*
Chicken	520	4.4	17	0	30	180	0.8	-
Fish	500	3.3	85	0	5	90	3.7	-
GroundBeef	860	0.3	82	0	75	350	2.3	-
FriedBeans	600	3.4	10	30	3	0	0.9	1.6667
Lettuce	50	0.5	6	0	0	0	0.75	-
Potatoes	460	2.2	10	70	0	0	0.4	0.3043
Milk2pct	240	0.2	16	22	10	20	0.83	1.5000

Table 1.

1.5 Optimal Results

When the LP model is run in CPLEX, the following optimal solution is produced:

Food Item	Quantity (lbs)
Fired Beans	1.6667
Potatoes	0.3043
Milk (2%)	1.5000
Optimal Cost / Serving	\$2.8668

Table 2 confirms that the food suggested by the optimal solution satisfies all the values set out for the nutritional requirements.

Nutrient	Achieved	Required
Calories	1,500 kcal	1,500 – 2,000 kcal
Iron	6.64 mg	≥ 5 mg
Fat	20.00 g	20 – 60 g
Protein	43.71 g	≥ 30 g
Carbohydrates	104.30 g	≥ 40 g
Cholesterol	30.00 mg	≤ 30 mg

Table 2.

1.6 Interpretation of Results

The menu proposed from the optimal solution costs \$2.87 per serving and contains the following three food items: Fried Beans, Potatoes, and Milk (2%). While the solution is mathematically correct and adheres to all six outlined constraints, the narrow food selection would warrant a sufficient explanation to the budget committee.

At optimality, there are two binding constraints that are restricting options: the calorie lower bound, which is being exactly met at 1,500 kcal, and the cholesterol limit, which is already being met. The cholesterol limit has the most significant influence over which foods are selected for the menu, as foods like Chicken and Ground Beef cannot be added as they would exceed the cholesterol limit significantly (180mg/lbs and 350mg/lbs, respectively).

Additionally, 1.5 lbs of milk has been selected (20lbs * 1.5lbs), utilising the entire budget for cholesterol and leaving no room for any other protein-rich food. This is why four out of the seven available foods do not appear in the solution at all, with higher-protein options effectively off the table, the model selects the cheapest remaining sources of calories and carbohydrates (Potatoes at \$0.40/lbs and Fried Beans at £0.90/lbs) to meet the nutritional requirements. Therefore, reducing the overall cost of the menu would require a relaxation of the cholesterol guidelines, where even a modest increase in the allowable limit would allow for the inclusion of cheaper protein sources.

1.7 Sensitivity Analysis

1.7.1 Reduced Costs (from CPLEX Output)

Table 3 presents the reduced costs and sensitivity ranges that are returned from running the model in CPLEX. The reduced cost of a non-basic variable ($x^* = 0$) shows how much the objective would worsen per pound if that food were forced into the solution. A reduced cost of zero indicates a basic variable that is currently selected.

Food Item	X^* (lbs)	Reduced Cost (\$)	Sensitivity Ranges
Chicken	0	2.3213	$[-0.0777, 0.0135]$ $[0, \infty]$
Fish	0	5.5128	$[-0.125, 0.0189]$ $[0, \infty]$
GroundBeef	0	3.2880	$[-0.05, 0.0084]$ $[0, \infty]$
FriedBeans	1.6667	0	$[-\infty, 1.6667]$ $[1.6667, \infty]$
Lettuce	0	0.7065	$[-2.7906, 2.8]$ $[0, \infty]$
Potatoes	0.3043	0	$[-\infty, 0.3043]$ $[0.3043, \infty]$
Milk2pct	1.5000	0	$[-\infty, 1.5]$ $[1.5, \infty]$

Table 3.

From the above table, Fish has the highest reduced cost at \$5.51, making it the most expensive food to introduce to the optimal solution relative to its nutritional contribution. Ground Beef (\$3.29) and Chicken (\$2.32) follow on from this and are only excluded primarily due to the cholesterol constraints outlined earlier. Lettuce (\$0.71) would be an inexpensive option to introduce, but it provides negligible nutritional value per pound, so it remains excluded from the optimal solution.

The sensitivity range for Fried Beans shows its cost coefficient could rise from \$0.90 to \$1.6667 per pound before it would leave the optimal basis, displaying an 85% cost increase before the solution changes. This suggests that Fried Beans is a robust choice even under price inflation.

2.0 Problem 2: Dr J's Assignment Problem

2.1 Problem Overview

Twenty-eight engineering design students must each be assigned to exactly one of eight projects (A, ED, EZ, G, H1, H2, RB, and SC), where each project must contain between three and four students. Students have submitted their ranked preferences (rank 1 = most preferred, rank 8 = least preferred) with the objective to minimise the total preference rank score across all assignments and maximise aggregate student satisfaction.

This is a Binary Integer Program (BIP) problem where the decision variables are binary (a student is either assigned to a project or not), making fractional LP relaxation invalid. To find the optimal solution, CPLEX utilises a Branch and Bound method, where it will search through all possible solutions without explicitly examining every single one.

2.2 Mathematical Formula

1.2.1 Decision Variables

$x_{\{i,j\}} \in \{0, 1\}$: equals 1 if student i is assigned to project j , otherwise 0.

1.2.2 Objective Function – Minimise Total Preference Rank

Minimise $Z = \sum_i \sum_j \text{pref}[i][j] \cdot x_{\{i,j\}}$ for all $i \in \text{Students}$, $j \in \text{Projects}$

1.2.3 Subject to

Each student assigned to exactly one project: $\sum x_{\{i,j\}} = 1 \quad \forall i$

Each project receives 3-4 students: $3 \leq \sum_i x_{\{i,j\}} \leq 4 \quad \forall j$

Binary restriction: $x_{\{i,j\}} \in \{0, 1\}$

2.3 OPL Model Code

The script provided for the .dat file contains an OUTPUT block that writes the results directly into an openable .xlsx file in Excel, as per the requirements for part (c) of the question.

2.3.1 Assignment.mod

```
/******
 * OPL 22.1.2.0 Model
 * Author: wilso
 * Creation Date: Mar 22, 2026 at 11:57:05 AM
 *****/

//sets and parameters
{string} S = ...; //set of students, indexed by i
{string} P = ...; //set of projects, indexed by j
int pref[S][P] = ...; //preference rank of student i for project j (1 = most preferred)
int minGroup = ...; //minimum number of students per project
int maxGroup = ...; //maximum number of students per project

//decision variables
dvar boolean x[S][P]; //1 if student i is assigned to project j, else 0

//objective function - minimise total preference rank (maximises satisfaction)
minimize sum(i in S, j in P) pref[i][j] * x[i][j];

//constraints
subject to {
    //each student must be assigned to exactly one project
    forall(i in S)
        ctOneProject: sum(j in P) x[i][j] == 1;

    //each project must receive between minGroup and maxGroup students
    forall(j in P) {
        ctMinSize: sum(i in S) x[i][j] >= minGroup;
        ctMaxSize: sum(i in S) x[i][j] <= maxGroup;
    }
}
```

2.3.2 Assignment.dat

```
/******
 * OPL 22.1.2.0 Data
 * Author: wilso
 * Creation Date: Mar 22, 2026 at 11:57:05 AM
 *****/

S = {"Allen", "Black", "Chung", "Clark", "Connors", "Cumming",
     "Demming", "Eng", "Farmer", "Forest", "Goodman", "Harris",
     "Holmes", "Johnson", "Knorr", "Manheim", "Morris", "Nathan",
     "Neuman", "Patrick", "Rollins", "Schuman", "Silver", "Stein",
     "Stock", "Truman", "Wolman", "Young"};

P = {"A", "ED", "EZ", "G", "H1", "H2", "RB", "SC"};

//preference rank matrix [student][project] - 1 = most preferred
pref = [
    [1, 3, 4, 7, 7, 5, 2, 6], // Allen
    [7, 4, 2, 5, 5, 8, 1, 3], // Black
    [7, 3, 4, 1, 1, 7, 6, 4], // Chung
    [8, 7, 1, 2, 2, 4, 6, 4], // Clark
    [8, 7, 1, 3, 3, 5, 6, 2], // Connors
    [7, 8, 5, 2, 2, 4, 6, 1], // Cumming
```

Author: Andrew Wilson (2417549)

```
[2, 5, 4, 6, 6, 1, 3, 8], // Demming
[5, 8, 3, 1, 1, 7, 4, 6], // Eng
[8, 7, 6, 2, 2, 1, 4, 5], // Farmer
[7, 8, 2, 5, 5, 1, 3, 4], // Forest
[8, 7, 2, 4, 4, 6, 1, 3], // Goodman
[5, 8, 6, 3, 3, 1, 2, 7], // Harris
[7, 8, 5, 2, 2, 4, 6, 1], // Holmes
[2, 2, 5, 7, 7, 6, 4, 1], // Johnson
[8, 5, 1, 2, 2, 6, 7, 4], // Knorr
[5, 8, 3, 1, 1, 4, 7, 6], // Manheim
[8, 5, 4, 6, 6, 3, 1, 2], // Morris
[4, 8, 5, 6, 6, 3, 1, 2], // Nathan
[8, 5, 4, 6, 6, 3, 1, 2], // Neuman
[1, 8, 6, 4, 4, 2, 3, 7], // Patrick
[7, 3, 4, 1, 1, 8, 6, 5], // Rollins
[4, 8, 3, 5, 5, 1, 2, 7], // Schuman
[5, 8, 4, 1, 1, 3, 6, 7], // Silver
[7, 4, 2, 5, 5, 8, 1, 3], // Stein
[5, 2, 1, 6, 6, 8, 4, 3], // Stock
[6, 3, 2, 7, 7, 5, 1, 4], // Truman
[7, 8, 5, 2, 2, 4, 6, 1], // Wolman
[1, 3, 4, 7, 7, 6, 2, 5] // Young
];

minGroup = 3;
maxGroup = 4;

//Connection and Write statements for the Excel file
SheetConnection sheet("assignment_results.xlsx");

//column headers - project names across row 1 (cols B to I)
P to SheetWrite(sheet, "Sheet1!B1:I1");

//row labels - student names down column A (rows 2 to 29)
S to SheetWrite(sheet, "Sheet1!A2:A29");

//assignment matrix (students x projects)
x to SheetWrite(sheet, "Sheet1!B2:I29");
```

2.4 Optimal Assignment Results

Table 4 below presents the complete optimal assignment for all 28 students. Colour coding:

green = 1st choice, orange = 2nd choice, red = 3rd choice

Student	Project	Preference Rank
Allen	A	1
Black	RB	1
Chung	G	1
Clark	EZ	1
Conners	EZ	1
Cumming	SC	1
Demming	H2	1
Eng	H1	1
Farmer	H1	2
Forest	H2	1
Goodman	EZ	2
Harris	H2	1
Holmes	SC	1
Johnson	ED	2
Knorr	EZ	1
Manheim	G	1
Morries	RB	1
Nathan	RB	1
Neuman	SC	2
Patrick	A	1
Rollins	H1	1
Schuman	H2	1
Silver	G	1
Stein	RB	1
Stock	ED	2
Truman	ED	3
Wolman	SC	1
Young	A	1

Table 4.

2.5 Summary Statistics

Metric	Value
Optimal total preference rank score	35
Students assigned 1 st choice	22 / 28 (78.6%)
Students assigned 2 nd choice	5 / 28 (17.9%)
Students assigned 3 rd choice	1 / 28 (3.6%)
Students within top-3 preference	28 / 28 (100%)

2.6 Project Group Composition

Table 5 shows the composition of each of the project groups, with all eight groups satisfying the 3 - 4 student constraint.

A	ED	EZ	G	H1	H2	RB	SC
Allen	Johnson	Clark	Chung	Eng	Demming	Black	Cumming
Patrick	Stock	Conners	Manheim	Farmer	Forest	Morris	Holmes
Young	Truman	Goodman	Silver	Rollins	Harris	Nathan	Neuman
		Knorr			Schuman	Stein	Wolman

Table 5.

2.7 Interpretation of Results

The model created achieves an optimal total rank score of 35 by assigning 22 students to their first group choice, five to their second, and one student to their third choice (Truman). Truman’s third choice assignment is a result of the group size constraints, with the group ED requiring three students, but most students did not rank it highly. By the time the higher-demand projects were optimally filled, Truman’s preferred options of RB and EZ were already at capacity, leaving ED as the only suitable placement. Across the remaining projects, students are placed in their first or second choices. The fact that every student lands within their top three preference is a meaningful outcome, as students who work on projects they have expressed an interest in tend to engage more effectively and produce better results than those assigned at random. As previously mentioned, the results from running the configurations associated with this problem are written directly into an .xlsx file, producing a labelled matrix with project names as column headers and student names as row labels.

3.0 Problem 3: Shortest Time to Walk

3.1 Problem Overview

The Ankaful Hospital campus is represented as a weighted network consisting of 14 facility nodes connected by 25 undirected walking paths, each labelled with travel time in minutes. The objective is to find the minimum-time walking route from the Hospital Administration Block (node 3) to the Nightingale Ward (node 8).

This represents a Shortest-Path problem that can be solved using a Linear Programming network flow formulation. The created model uses the tuple arc data structure and flow balance constraints to ensure that the flow travels along a single connected path from start to finish, guaranteeing the minimum time required.

3.2 Network Structure

The network consists of 14 nodes and 25 undirected arcs. Each undirected arc is modelled as two directed arcs, representing forward and reverse, to keep the flow balance constraints valid, giving 50 directed arcs in total. Only nodes 3 and 8 are named in the problem, while all other nodes are referenced by their integer index. Table 6 lists all the arcs and their travel times as given in the problem diagram.

Arc	Time (min)	Arc	Time (min)
1 – 2	15	6 – 10	25
1 – 3	5	6 – 12	35
1 – 4	4	7 – 8	15
2 – 4	35	8 – 11	15
2 – 5	20	8 – 14	50
3 – 4	10	9 – 10	10
3 – 6	15	9 – 11	15
4 – 5	30	10 – 13	25
4 – 6	15	11 – 13	30
4 – 9	45	11 – 14	20
5 – 7	15	12 – 13	50
5 – 8	25	13 – 14	45
5 – 9	10		

Table 6.

3.3 Mathematical Formula

3.3.1 Decision Variables

$x_a \geq 0$: flow along arc a . Takes value 1 if the arc is on the shortest path, 0 otherwise. The capacity constraint $x_a \leq 1$ enforces binary path selection.

3.3.2 Objective Function – Minimise Total Walking Time

Minimise $Z = \sum(a \in A) a.time * x[a]$

3.3.3 Flow Balance Constraints

For node 3 (source): flow out - flow in = 1

For node 8 (sink): flow in - flow out = 1

For all other nodes (1,2,4-7,9-14): flow in - flow out = 0

Capacity: $x_a \leq 1$ for all arcs a

These constraints ensure exactly one unit of flow travels along a single connected path from node 3 to node 8.

3.4 OPL Model Code

The model uses a tuple arc data structure with integer node indices matching the diagram shown in the problem, with the travel time embedded in the tuple so the ordering of arcs in the .dat file does not affect correctness. The source and sink values are declared as data parameters to allow for origin-destination pairing to be solved by changing two values in the .dat file.

3.4.1 Shortestpath.mod

```
/******
 * OPL 22.1.2.0 Model
 * Author: wilso
 * Creation Date: Mar 22, 2026 at 1:04:55 PM
 *****/

//user-defined data structure - arc carries its own travel time
tuple arc {
    int orig; //origin node of the arc
    int dest; //destination node of the arc
    int time; //walking time (minutes) along this arc
}

//sets and parameters
int nNodes = ...; // read from .dat
range N = 1..nNodes; // derived in the model
{arc} A = ...; //set of directed arcs (each undirected path split into two)
int source = ...; //origin node - Hospital Administration Block (3)
int sink = ...; //destination node - Nightingale Ward (8)

//decision variables
dvar float+ x[A]; //flow on arc a: 1 if on the shortest path, 0 otherwise

//objective function - minimise total walking time
minimize sum(a in A) a.time * x[a];

//constraints
subject to {
    //flow balance at every node
    forall(k in N)
        ctFlow: sum(a in A: a.orig == k) x[a] - sum(a in A: a.dest == k) x[a]
            == (k == source ? 1 : (k == sink ? -1 : 0));

    //capacity: at most one unit of flow on any arc
    forall(a in A)
        ctCapacity: x[a] <= 1;
}
```

3.4.2 Shortestpath.dat

```
/******
 * OPL 22.1.2.0 Data
 * Author: wilso
 * Creation Date: Mar 22, 2026 at 1:04:55 PM
 *****/

//Network has 14 nodes and 25 undirected arcs.
//Each undirected arc is represented as two directed arcs
//((forward + reverse) so flow balance constraints are valid.
//Arc time is embedded in the tuple so no separate array is needed.

nNodes = 14; //14 facility nodes

//each arc: <origin, destination, time_in_minutes>
//undirected paths represented as two directed arcs
A = {
```

Author: Andrew Wilson (2417549)

```
< 1, 2, 15>, < 2, 1, 15>, // node 1 <-> node 2
< 1, 3, 5>, < 3, 1, 5>, // node 1 <-> node 3
< 1, 4, 4>, < 4, 1, 4>, // node 1 <-> node 4
< 2, 4, 35>, < 4, 2, 35>, // node 2 <-> node 4
< 2, 5, 20>, < 5, 2, 20>, // node 2 <-> node 5
< 3, 4, 10>, < 4, 3, 10>, // node 3 <-> node 4
< 3, 6, 15>, < 6, 3, 15>, // node 3 <-> node 6
< 4, 5, 30>, < 5, 4, 30>, // node 4 <-> node 5
< 4, 6, 15>, < 6, 4, 15>, // node 4 <-> node 6
< 4, 9, 45>, < 9, 4, 45>, // node 4 <-> node 9
< 5, 7, 15>, < 7, 5, 15>, // node 5 <-> node 7
< 5, 8, 25>, < 8, 5, 25>, // node 5 <-> node 8
< 5, 9, 10>, < 9, 5, 10>, // node 5 <-> node 9
< 6, 10, 25>, <10, 6, 25>, // node 6 <-> node 10
< 6, 12, 35>, <12, 6, 35>, // node 6 <-> node 12
< 7, 8, 15>, < 8, 7, 15>, // node 7 <-> node 8
< 8, 11, 15>, <11, 8, 15>, // node 8 <-> node 11
< 8, 14, 50>, <14, 8, 50>, // node 8 <-> node 14
< 9, 10, 10>, <10, 9, 10>, // node 9 <-> node 10
< 9, 11, 15>, <11, 9, 15>, // node 9 <-> node 11
<10, 13, 25>, <13, 10, 25>, // node 10 <-> node 13
<11, 13, 30>, <13, 11, 30>, // node 11 <-> node 13
<11, 14, 20>, <14, 11, 20>, // node 11 <-> node 14
<12, 13, 50>, <13, 12, 50>, // node 12 <-> node 13
<13, 14, 45>, <14, 13, 45> // node 13 <-> node 14
};

source = 3; //Hospital Administration Block
sink   = 8; //Nightingale Ward
```

3.5 Optimal Solution

After running the model, CPLEX returns an optimal objective value of 64 minutes, as shown in Table 7.

From Node	To Node	Time (min)
Node 3 (source)	Node 1	5
Node 1	Node 4	4
Node 4	Node 5	30
Node 5	Node 8 (sink)	25
Total	Path: 3 → 1 → 4 → 5 → 8	64

Table 7.

3.6 Interpretation of Results

From the results produced by CPLEX, it is confirmed that the optimal walking time from the Hospital Administration Block (node 3) to Nightingale Ward (node 8) is 64 minutes. All four arcs on this path carry a flow value of 1 and a reduced cost of 0, confirming they form the unique shortest path through the network. The two closest alternatives both take 65 minutes, with one travelling directly from node 3 to node 4 ($3 \rightarrow 4 \rightarrow 5 \rightarrow 8 = 10 + 30 + 25$) and the other via node 2 ($3 \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow 8 = 5 + 15 + 20 + 25$). The sensitivity ranges from CPLEX reinforces the robustness of this solution, with each arc on the optimal path having a sensitivity range of $[-\infty, 1]$, meaning each arc time could increase by up to 1 minute before an alternative route would become suitable. The arc $3 \rightarrow 4$ has a reduced cost of 1, which independently confirms that the direct route from node 3 to node 4 is the nearest competitor. From a hospital operations perspective, this result provides a quantitative basis for patient escort planning, staff wayfinding, and emergency response routing. If required, this model can be reused by changing the source and sink values in the .dat file to allow for any origin-destination pair across the network to be solved without modifying the model itself.